

Doremi Python vol 2

도레미 파이썬 vol 2

Python 심화 문법

목차 Python 심화 문법

1. f-string 포매팅
2. List Comprehension
3. Sorted
4. Enumerate
5. Lambda
6. if __name__ == "__main__"

1. f-string 포맷팅

f-string 포매팅

- Python 3.6 이상부터 기존의 print 보다 더 상세하게 출력할 수 있는 방법
- print와 함께 **f** 그리고 **중괄호{}**를 이용하여 표현
 - f를 먼저 선언
 - 문자열(" ") 안에 중괄호{}와 변수명 작성

```
hour = 9  
minute = 15
```

```
word_1 = "오전"
```

```
print("현재 시간은", word_1, hour, "시",  
minute, "분 입니다.")
```

```
hour = 9  
minute = 15
```

```
word_1 = "오전"
```

```
print(f"현재 시간은 {word_1} {hour}시  
{minute}분 입니다. ")
```

2. List Comprehension



List Comprehension 이란?

- Python에서 리스트를 선언하는 방법
- 리스트를 생성할 때, 조건문과 반복문을 사용한 여러 줄로 된 코드를 **한 줄로 줄일 수 있음**
- 리스트를 생성하는 대괄호 안에 조건문과 반복문을 사용하여 리스트를 생성할 수 있음

```
# 0부터 99까지의 숫자 중에서 짝수만 저장된 리스트를 생성  
num_list = [num for num in range(100) if num%2 == 0]
```

List Comprehension의 동작 순서

Case 1. 반복문만 이용한 사례

```
new_list = [ <표현식> for <변수> in <시퀀스> ]
```

(3) 변환된 정보를
newList에 리스트로 저장

(2) 변수(자료)를
지정한 표현식으로 변환

(1) 시퀀스 자료형에서 변수(자료)를
하나씩 가져오기

```
new_list = [ num*10 for num in range(10) ]
```

(3) 변환된 정보를
newList에 리스트로 저장

(2) 가져온 변수에
곱하기 10 연산

(1) 0~9 까지의 시퀀스 자료형에서
하나씩 가져오기

newList 결과 : [0, 10, 20, 30, 40, 50, 60, 70, 80, 90]



0~99까지 숫자를 저장한 리스트 생성

```
num_list = []  
  
for num in range(100) :  
    num_list.append(num)  
  
print(num_list)
```

실행 결과

[0, 1, 2, ..., 98, 99]

```
num_list = [num for num in range(100)]  
  
print(num_list)
```

실행 결과

[0, 1, 2, ..., 98, 99]

List Comprehension의 동작 순서

Case 2. 반복문과 조건문(if)을 이용한 사례

```
new_list = [ <표현식> for <변수> in <시퀀스> if <조건> ]
```

(4) 변환된 정보를
newList에
리스트로 저장

(3) 조건의 결과 True인
변수(자료)를
지정한 표현식으로 변환

(1) 시퀀스 자료형에서 변수(자료)를
하나씩 가져오기

(2) 변수가 지정한 조건에
맞는지 확인

```
new_list = [ str(num) for num in range(10) if num%2 == 0 ]
```

(4) 변환된 정보를
newList에
리스트로 저장

(3) 조건의 결과 True인
변수를 문자열로 변환

(1) 0~9 까지의 시퀀스 자료형에서
하나씩 가져오기

(2) 가져온 변수에서 2로 나눈 나머지가
0인지 확인

new_list 결과 : ['0', '2', '4', '6', '8']



특정 텍스트만 추출하여 저장하는 방법

```
region_list = ["수원(경기)",  
"군산(전북)", "포항(경북)",  
"경주(경북)", "익산(전북)", "강릉(강원)"  
]
```

```
gyeonggi_list = []
```

```
for region in region_list :  
    if "경기" in region :  
        gyeonggi_list.append(region)
```

```
print(gyeonggi_list)
```

```
region_list = ["수원(경기)",  
"군산(전북)", "포항(경북)",  
"경주(경북)", "익산(전북)", "강릉(강원)"  
]
```

```
gyeonggi_list =  
[region for region in region_list  
if "경기" in region]
```

```
print(gyeonggi_list)
```

List Comprehension의 동작 순서

Case 3. 반복문과 조건문(if-else)을 이용한 사례

- if 와 함께 **else** 가 있어야 함
- **Case 2와 순서가 다름**

```
new_list = [ <표현식> if <조건> else <표현식2> for <변수> in <시퀀스> ]
```

(4) 변환된 정보를
newList에
리스트로 저장

(3) **조건**의 결과
True인 변수(자료)를
지정한 표현식으로 변환

(2) 변수가
지정한 조건에
맞는지 확인

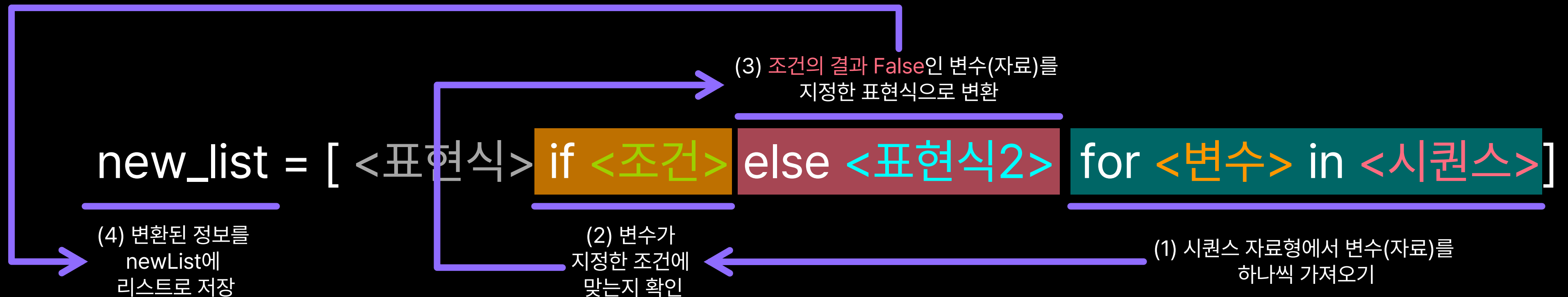
(1) 시퀀스 자료형에서 변수(자료)를
하나씩 가져오기

[조건이 참(**True**)일 경우]

List Comprehension의 동작 순서

Case 3. 반복문과 조건문(if-else)을 이용한 사례

- if 와 함께 **else** 가 있어야 함
- **Case 2와 순서가 다름**



[조건이 거짓(False)일 경우]



물건의 가격에 따라 할인을 적용

```
original_prices = [800, 1200, 950, 1100]
discounted_prices = []

for price in original_prices :
    if price >= 1000 :
        discounted_prices.append(int(price * 0.9))
    else :
        discounted_prices.append(price)

print(original_prices)

print(discounted_prices)
```

실행 결과

```
[800, 1200, 950, 1100]
[800, 1080, 950, 990]
```

```
original_prices = [800, 1200, 950, 1100]

discounted_prices = [
    int(price * 0.9) if price >= 1000 else price
for price in original_prices
]

print(original_prices)

print(discounted_prices)
```

실행 결과

```
[800, 1200, 950, 1100]
[800, 1080, 950, 990]
```

3. Sorted



리스트를 정렬하는 방법 (1)

- Python에서 리스트를 정렬하는 방법은 **sort 메소드**를 사용
- sort 메소드는 원본 리스트를 정렬
- sort 메소드를 사용한 명령어를 저장하면 None이 저장됨

```
my_list = [5, 2, 3, 1, 4]
my_list.sort()

print(my_list)
```

```
my_list = [5, 2, 3, 1, 4]
sort_list = my_list.sort()

print(my_list)

print(sort_list)
```



리스트를 정렬하는 방법 (2)

- Python에서 리스트를 정렬하는 또다른 방법은 **sorted 함수**를 사용
- sorted 함수는 복사본 리스트를 정렬
- sorted 함수를 사용한 명령어를 저장하면 **정렬된 리스트가 저장됨**

```
my_list = [5, 2, 3, 1, 4]
sort_list = sorted(my_list)

print(my_list)

print(sort_list)
```




sort 메소드 vs sorted 함수

	sort 메소드	sorted 함수
결과	원본 리스트를 정렬	복사본 리스트를 정렬
사용법 (오름차순)	list. sort ()	result = sorted (list)
내림차순 정렬	list. sort (reverse = True)	result = sorted (list, reverse = True)
장점	sorted 함수보다 계산 효율이 미세하게 좋음	원본 리스트를 유지할 수 있음

4. Enumerate



enumerate 이란?

- 순서가 있는 자료형을 대상으로 **인덱스(순서)와 그 값을 전달하는 기능을 가진 함수**
- enumerate 함수를 print하면 내부 값을 알기 어려움
- list로 형변환하면 확인할 수 있음
- 시작 번호를 지정할 수 있음 (보통은 0부터 시작)

```
my_list = ["Spring", "Summer", "Fall", "Winter"]

print(enumerate(my_list))
# <enumerate object at 0x7f0127cc4948>

print(list(enumerate(my_list)))
# [(0, "Spring"), (1, "Summer"), (2, "Fall"), (3, "Winter")]

print(list(enumerate(my_list, start=2023)))
# [(2023, "Spring"), (2024, "Summer"), (2025, "Fall"), (2026, "Winter")]
```



enumerate 함수와 for문

- enumerate은 **for문과 함께 자주 사용**됨
- for문에서는 list 형변환 없이 바로 사용할 수 있음

```
my_list = ["elice", "rabbit", "clock", "queen"]
```

```
for idx, value in enumerate(my_list) :  
    print(idx)  
    print(value)
```

my_list의
인덱스 번호

my_list의 인덱스 번호에
해당하는 값



enumerate 함수 사용 예시 (1)

순서가 있는 자료형(리스트, 딕셔너리, 문자열 등)을 대상으로 다양하게 사용할 수 있음

```
grades = [85, 92, 78, 90, 88]

for index, grade in enumerate(grades, start=1) :
    print(f"학생 {index}의 성적: {grade}점")
```

실행 결과

학생 1의 성적: 85점
학생 2의 성적: 92점
학생 3의 성적: 78점
학생 4의 성적: 90점
학생 5의 성적: 88점



enumerate 함수 사용 예시 (2)

순서가 있는 자료형(리스트, **딕셔너리**, 문자열 등)을 대상으로 다양하게 사용할 수 있음

```
temperatures = {  
    "서울" : 28,  
    "부산" : 30,  
    "대구" : 29,  
    "광주" : 31  
}  
  
for place, (city, temp) in enumerate(temperatures.items(), start=1):  
    print(f "{place}. {city}의 온도: {temp}도")
```

실행 결과

1. 서울의 온도: 28도
2. 부산의 온도: 30도
3. 대구의 온도: 29도
4. 광주의 온도: 31도



enumerate 함수 사용 예시 (3)

순서가 있는 자료형(리스트, 딕셔너리, 문자열 등)을 대상으로 다양하게 사용할 수 있음

```
text = "안녕하세요, 파이썬!"  
  
for index, char in enumerate(text, start=1) :  
    print(f"{index}번째 글자: {char}")
```

실행 결과

1번째 글자: 안
2번째 글자: 녕
3번째 글자: 하
4번째 글자: 세
5번째 글자: 요
...
11번째 글자: !

5. Lambda



Lambda 란?

Python에서 간단한 함수를 작성할 때 사용되는 **익명 함수**

lambda <매개변수> : <표현식>

함수의 매개변수 역할

매개변수를 이용하여
표현할 표현식(return 값)

```
square = lambda x: x ** 2
```

```
result = square(5)
```

```
print(result)
```

```
result = (lambda x: x ** 2) (5)
```

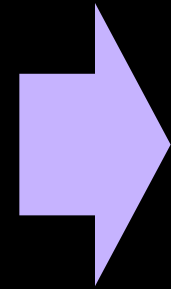
```
print(result)
```

Lambda 특징

- 간결하고 한 줄로 표현할 수 있으며 별도의 함수 정의 없이도 간단한 기능을 표현할 수 있음
- 복잡한 로직이나 긴 표현식을 사용하기에는 적합하지 않을 수 있음

```
add = lambda x, y : x*y+2
```

```
print(add(3, 4))
```



```
add = lambda 3, 4 : 3*4+2
```

```
print(add(3, 4))
```



일반 함수와 lambda 함수 비교

상품 가격과 수량을 받아 세금을 포함한 총 주문 가격을 계산하는 함수

```
def calculate_total_price(item_price, quantity) :  
    tax_rate = 0.1  
    total_price = item_price * quantity  
    total_with_tax = total_price * (1 + tax_rate)  
  
    return total_with_tax  
  
# 각 물건의 가격  
item_price = 100  
  
# 주문 수량  
quantity = 52  
  
total = calculate_total_price(item_price, quantity)  
  
print(total)
```

```
calculate_total_price =  
lambda item_price, quantity : (item_price * quantity) * 1.1  
  
# 각 물건의 가격  
item_price = 100  
  
# 주문 수량  
quantity = 52  
  
total = calculate_total_price(item_price, quantity)  
  
print(total)
```



Lambda 심화 - map

- **map 함수**는 함수와 시퀀스 자료형을 인자로 받음
- 시퀀스 자료형의 자료 하나씩 함수를 적용
- map 함수의 결과를 그대로 출력하면 결과 확인이 어려움, list로 형변환을 해서 확인 (map과 list가 주로 같이 사용됨)

```
names = ["elice", "rabbit", "clock", "queen"]  
  
upper_names = map(str.upper, names)  
print(upper_names)  
  
upper_list = list(upper_names)  
print(upper_list)
```

실행 결과

```
<map object at 0x7fe8a4263160>  
["ELICE", "RABBIT", "CLOCK", "QUEEN"]
```




Lambda 심화 - map

- **map 함수**의 자리에 Lambda 함수 사용 가능
- 아래의 두 예제는 같은 Lambda 함수를 사용했지만 Lambda 함수 위치와 list 형변환 위치가 다름 (정해진 규칙은 없음!)

```
square = lambda x: x ** 2

num_list = [10, 11, 12, 13, 14, 15]

map_function = map(square, num_list)

print(list(map_function))
```

```
num_list = [10, 11, 12, 13, 14, 15]

map_function =
list(map(lambda x: x ** 2, num_list))

print(list(map_function))
```

6. `if __name__ == "__main__":`



모듈 (Module) 이란?

- 코드의 길이가 길어지는 상황에서 모든 함수, 변수를 구현하는 것은 어려움
 - 누군가가 만든 함수와 변수 등을 활용
- 모듈 (Module) = 특정 목적을 가진 함수, 자료들의 모임

```
# cal.py
```

```
def plus(a, b) :  
    c = a + b  
  
    return c
```

```
# main.py
```

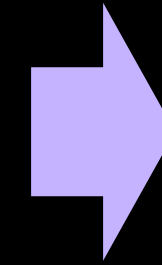
```
import cal
```

```
print(cal.plus(3, 4))
```



main.py 실행 결과

```
# main.py  
import cal
```



실행 결과

6
4

```
# cal.py  
def plus(a, b) :  
    c = a + b  
    return c  
  
def sub(a, b) :  
    c = a - b  
    return c
```

```
print(plus(1, 5))  
print(sub(5, 1))
```

cal.py를 직접 실행했을 때에만
해당 코드를 실행하고 싶다면
어떻게 해야할까?

Python 파일 실행 = 모든 코드 실행

- 특정 프로그래밍 언어(e.g. C, JAVA)에서 파일을 실행하면 main이라는 함수부터 시작
 - 시작점이 분명함
- Python은 main이라는 함수가 없어서 모든 코드(특히 들여쓰기가 되지 않은 코드)를 실행함
 - 프로그램을 구성할 때, 의도하지 않은 코드가 실행될 수 있음
 - 프로그램 동작 순서를 구성하기 어려움

```
int main(void) {  
    printf("%d\n", plus(1, 5));  
    printf("%d\n", sub(5, 1));  
    return 0;  
}
```

프로그램을 시작하면
해당 부분부터 시작

```
int plus(int num1, int num2) {  
    return num1 + num2;  
}
```

```
int sub(int num1, int num2) {  
    return num1 - num2;  
}
```

```
def plus(a, b) :  
    c = a + b  
    return c
```

```
def sub(a, b) :  
    c = a - b  
    return c
```

```
print(plus(1, 5))  
print(sub(5, 1))
```

가장 위에서부터
순서대로 시작



Python 파일 실행 = 모든 코드 실행

if `__main__` == `"__main__"` 을 이용하면 시작점을 지정할 수 있음

```
def plus(a, b) :  
    c = a + b  
    return c
```

```
def sub(a, b) :  
    c = a - b  
    return c
```

```
if __name__ == "__main__" :  
    print(plus(1, 5))  
    print(sub(5, 1))
```

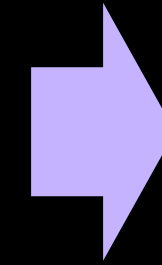
if __main__ == "__main__" 사용 결과

```
# main.py  
import cal
```

```
# cal.py  
def plus(a, b) :  
    c = a + b  
    return c
```

```
def sub(a, b) :  
    c = a - b  
    return c
```

```
if __name__ == "__main__" :  
    print(plus(1, 5))  
    print(sub(5, 1))
```



실행 결과

(아무것도 출력되지 않음)

(문제) 다음 코드의 실행 순서를 고르세요

```
import sys
```

```
def sum(a, b) :  
    c = a + b  
    return c
```

?

```
def main() :  
    print("3 더하기 6은?")  
  
    result = sum(3, 6)  
    print(result)
```

?

```
if __name__ == "__main__" :  
    sys.exit(main())
```

?

sys.exit() 는 프로그램을 종료할 때, 오류가 발생했는지 알려주는 함수

(정답) 다음 코드의 실행 순서를 고르세요

```
import sys
```

```
def sum(a, b) :  
    c = a + b  
    return c
```

3

```
def main() :  
    print("3 더하기 6은?")  
  
    result = sum(3, 6)  
    print(result)
```

2

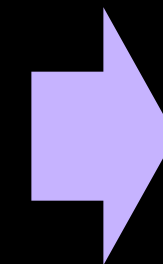
```
if __name__ == "__main__" :  
    sys.exit(main())
```

1

- sum 함수 실행
- 3, 6 계산 결과 반환

- main 함수 실행
- sum 함수 호출 (3, 6 전달)

- main 함수 호출



실행 결과

3 더하기 6은?

9